

ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΑΣΚΗΣΗ 1

(Classification Problem)

Μάθημα: Υπολογιστική Νοημοσύνη

ΜΠΟΥΖΟΥΚΑΣ ΚΩΝΣΤΑΝΤΙΝΟΣ-ΛΕΥΤΕΡΗΣ 5302

ΝΙΚΟΛΑΚΟΠΟΥΛΟΣ ΑΡΙΣΤΕΙΔΗΣ 5308

Ημερομηνία: 7/1/2025

!!! Εντολές μεταγλώττισης και δημιουργίας εκτελέσιμου

PT2:

```
gcc PT2.c -o PT2 -lm
```

```
execute Command : ./PT2
```

PT3:

```
gcc PT3.c -o PT3 -lm
```

```
execute Command : ./PT3
```

classificationProblemExamples:

```
gcc classificationProblemExamples.c -o CPE
```

```
execute Command : ./CPE
```

1. Δημιουργία παραδειγμάτων

Για την δημιουργία παραδειγμάτων γράψαμε ένα script το οποίο δημιουργεί 2 αρχεία (.csv) ένα με 4000 παραδείγματα για το σύνολο εκπαίδευσης και ένα ακόμα για το σύνολο ελέγχου.

Δημιουργεί τυχαίους αριθμούς μεταξύ του -1 και του 1 (x_1, x_2) και σύμφωνα με τις οδηγίες τους κατατάσσει στις κατηγορίες (C1,C2,C3,C4) και

αποθηκεύει το κάθε παράδειγμα (x_1, x_2, class) στο εκάστοτε αρχείο.

2. Υλοποίηση της άσκησης

1)

```
9  #define D 2
10 #define K 4
11 #define H1 20
12 #define H2 10
13 #define #define ACTIVATION_RELU 1
14 #define ACTIVATION_RELU 1
15 #define ACTIVATION_TANH 0
16 #define MIN_EPOCHS 800
17 #define MAX_ERROR 0.0001
18 #define LEARNING_RATE 0.001
19 #define N 4000
20 #define B 200
```

2) Για το φόρτωμα των συνόλων

χρησιμοποιήσαμε τα 2 προαναφερθέντα αρχεία και τα φορτώσαμε σε ένα FILE αντικείμενο

Παράλληλα δημιουργήσαμε ένα struct examples το οποίο κρατάει την πληροφορία για κάθε ζεύγος παραδείγματος σε έναν πίνακα και έναν ακόμα πίνακα την πληροφορία για την επιθυμητή έξοδο με 0 για κάθε θέση που δεν αντιστοιχεί στην κατηγορία του και 1 στην θέση που αντιστοιχεί.

Με την φόρτωση του αρχείου κάθε γραμμή δημιουργεί ένα "examples" και το αποθηκεύει σε έναν πίνακα από examples από όπου λαμβάνουμε

τα στοιχεία που χρησιμοποιούνται στην ροή του προγράμματος

3) Για τον καθορισμό του κατωφλίου και του ρυθμού μάθησης βλέπε εικόνα για ερώτημα 1.

Ορισμός απαιτούμενων πινάκων και δομών:

```
22 //Weights and Biases for each level
23 //Inp -> H1
24 double weights_inp_to_h1[D][H1];
25 double der_errors_w_inp_to_h1[D][H1];
26
27 double bias_h1[H1];
28 double der_errors_b_h1[H1];
29
30 double h1_total_inputs[H1];
31 double h1_outputs[H1];
32
33 //H1 -> H2
34 double weights_h1_to_h2[H1][H2];
35 double der_errors_w_h1_to_h2[H1][H2];
36
37 double bias_h2[H2];
38 double der_erros_b_h2[H2];
39
40 double h2_total_inputs[H2];
41 double h2_outputs[H2];
42
43 //H2 -> H3
44 double weights_h2_to_h3[H2][H3];
45 double der_errors_w_h2_to_h3[H2][H3];
46
47 double bias_h3[H3];
48 double der_erros_b_h3[H3];
49
50 double h3_total_inputs[H3];
51 double h3_outputs[H3];
52
53
54 //H3 -> Out
55 double weights_h3_to_out[H3][K];
56 double der_errors_w_h3_to_out[H3][K];
57
58 double bias_out[K];
59 double der_errors_b_out[K];
60
61 double outs[K];
62 double *outputs;
```

(Προφανώς για το PT2 υπάρχει ένα λιγότερο κρυμμένο επίπεδο)
Στην ουσία δημιουργούμε για μελλοντική χρήση

σε επίπεδο

1 Πίνακα για τα βάρη από το H-1 στο H

1 Πίνακα για τις παραγώγους των βαρών

1 Πίνακα για τις πολώσεις του H

1 Πίνακα για τις εισόδους του H

1 Πίνακα για τις εξόδους του H

Και για την τυχαία αρχικοποίηση των βαρών και των πολώσεων στο $[-1,1]$

```
327 //***** initialize weights and biases *****
328
329 //weights D to H1 and biases of H1
330 for(int i =0; i< H1;i++){
331     bias_h1[i] = make_random_in_range();
332     for(int j = 0;j<D;j++){
333         weights_inp_to_h1[j][i] = make_random_in_range();
334     }
335 }
336
337 //weights H1 to H2 and biases of H2
338 for(int i =0; i< H2;i++){
339     bias_h2[i] = make_random_in_range();
340     for(int j = 0;j<H1;j++){
341         weights_h1_to_h2[j][i] = make_random_in_range();
342     }
343 }
344
345 //weights H2 to H3 and biases of H3
346 for(int i =0; i< H3;i++){
347     bias_h3[i] = make_random_in_range();
348     for(int j = 0;j<H2;j++){
349         weights_h2_to_h3[j][i] = make_random_in_range();
350     }
351 }
352
353 //weights H3 to out and biases of out
354 for(int i =0; i< K;i++){
355     bias_out[i] = make_random_in_range();
356     for(int j = 0;j<H3;j++){
357         weights_h3_to_out[j][i] = make_random_in_range();
358     }
359 }
```

4)Υλοποίηση της forward_pass

```
182 double* forward_pass(double *x,int d,double *y,int k){
183
184     //From inputs to H1
185     for(int i =0;i<H1;i++){
186         double temp =0;
187         for(int j =0;j<d;j++){
188             temp += x[j] * weights_inp_to_h1[j][i];
189         }
190         temp+=bias_h1[i];
191         h1_total_inputs[i] = temp;
192         h1_outputs[i] = func_activation(temp,ACTIVATION_TANH);
193     }
194
195     //From H1 to H2
196     for(int i =0;i<H2;i++){
197         double temp =0;
198         for(int j =0;j<H1;j++){
199             temp += h1_outputs[j] * weights_h1_to_h2[j][i];
200         }
201         temp+=bias_h2[i];
202         h2_total_inputs[i] = temp;
203         h2_outputs[i] = func_activation(temp,ACTIVATION_RELU);
204     }
205
206     //From H2 to H3
207     for(int i =0;i<H3;i++){
208         double temp =0;
209         for(int j =0;j<H2;j++){
210             temp += h2_outputs[j] * weights_h2_to_h3[j][i];
211         }
212         temp+=bias_h3[i];
213         h3_total_inputs[i] = temp;
214         h3_outputs[i] = func_activation(temp,ACTIVATION_RELU);
215     }
216
217     //From H3 to Out
218     for(int i =0;i<k;i++){
219         double temp =0;
220         for(int j =0;j<H3;j++){
221             temp += h3_outputs[j] * weights_h3_to_out[j][i];
222         }
223         temp+=bias_out[i];
224         outs[i] = temp;
225     }
226
227     y = output_function(outs,k);
228
229     return y;
230 }
```

Η υλοποίηση της συνάρτησης πραγματοποιείται ως εξής,

Αρχικά η κάθε είσοδος δίνει πληροφορία σε κάθε νευρώνα του επόμενου επιπέδου όπου πολλαπλασιάζοντας με το βάρος της διαδρομής και προσθέτοντας την πόλωση έχουμε την είσοδο του νευρώνα όπου αποθηκεύεται κατάλληλα (για χρήση αργότερα) και με χρήση της συνάρτησης ενεργοποίησής (relu ή tanhu) παίρνουμε την έξοδο του νευρώνα.

Επαναλαμβάνουμε για κάθε κρυμμένο επίπεδο ενώ για την έξοδο ακολουθούμε τα ίδια βήματα έως την χρήση της συνάρτησης ενεργοποίησης όπου στην προκειμένη φάση χρησιμοποιούμε την συνάρτηση softmax (για να φτιάξουμε ποσοστά από τις εισόδους των νευρώνων εξόδου) όπου μας δίνει το διάνυσμα εξόδου (K_1, K_2, K_3, K_4) με τιμές μεταξύ του $(0,1)$ και $\sum(K) = 1$ ώστε να λαμβάνουμε ως κατηγορία την έξοδο με τιμή πλησιέστερη στο 1.

5)Υλοποίηση της backprop

```
232 void backprop(double *x, int d, double *t, int k){
233     //H3 -> K
234     double delta_out[k];
235     for(int i = 0;i<k;i++){
236         delta_out[i] = outputs[i] - t[i];
237         der_errors_b_out[i] = delta_out[i];
238         for(int j = 0;j<H3;j++){
239             der_errors_w_h3_to_out[j][i] = h3_outputs[j] * delta_out[i];
240         }
241     }
242
243     //H2 -> H3
244     double delta_H3[H3];
245
246     for(int i = 0;i<H2;i++){
247         double temp = 0;
248         for(int j= 0;j<k;j++){
249             temp += weights_h3_to_out[i][j]*delta_out[j];
250         }
251
252         delta_H3[i] = derivative_activation(h3_total_inputs[i],ACTIVATION_RELU)*temp;
253         der_erros_b_h3[i] = delta_H3[i];
254
255         for(int j= 0;j<H1;j++){
256             der_errors_w_h1_to_h2[j][i] = h2_outputs[j]*delta_H3[i];
257         }
258     }
259
260     //H1 -> H2
261     double delta_H2[H2];
262
263     for(int i = 0;i<H2;i++){
264         double temp = 0;
265         for(int j= 0;j<k;j++){
266             temp += weights_h2_to_h3[i][j]*delta_H3[j];
267         }
268
269         delta_H2[i] = derivative_activation(h2_total_inputs[i],ACTIVATION_RELU)*temp;
270         der_erros_b_h2[i] = delta_H2[i];
271
272         for(int j= 0;j<H1;j++){
273             der_errors_w_h1_to_h2[j][i] = h1_outputs[j]*delta_H2[i];
274         }
275     }
276
277     //Inp -> H1
278     double delta_H1[H1];
279     for(int i = 0;i<H1;i++){
280         double temp = 0;
281         for(int j = 0;j<H2;j++){
282             temp += weights_h1_to_h2[i][j]*delta_H2[j];
283         }
284
285         delta_H1[i] = derivative_activation(h1_total_inputs[i],ACTIVATION_TANH)*temp;
286         der_errors_b_h1[i] = delta_H1[i];
287
288         for(int j = 0;j<d;j++){
289             der_errors_w_inp_to_h1[j][i] = x[j] * delta_H1[i];
290         }
291     }
292 }
```

Αντίστροφα από την forwardpass για την backprop ξεκινάμε από τους νευρώνες εξόδου και υπολογίζουμε τα σφάλματα (δ) και τις μερικές παραγώγους βάρους σύνδεσης (και πολώσεων) σύμφωνα με τους τύπους για το αντίστροφο πέρασμα (Κ4 διαφάνειες μαθήματος) όπου και τα αποθηκεύουμε σε έναν πίνακα (για χρήση στην gradient descent). Επαναλαμβάνουμε μέχρι να φτάσουμε στο επίπεδο εισόδου.

6)Υλοποίηση gradient descent

δημιουργούμε για while(true) loop όπου και μέσα εκτελείται η βασική ροή του προγράμματος Και «σπάει» (break;) αν η τρέχουσα εποχή είναι μεγαλύτερη από 800 και το απόλυτο της διαφοράς του συνολικού σφάλματος εκπαίδευσης 2 διαδοχικών εποχών είναι μικρότερο του κατωφλίου τερματισμού.

Για N/B batches δημιουργούμε προσωρινούς πίνακες για αποθήκευση τιμών παραγώγων των βαρών (ώστε να μπορούμε να έχουμε την πληροφορία της τρέχουσας εποχής) και για κάθε batch (δηλ για B φορές και με αυτόν τον τρόπο επιτυγχάνουμε να μπορούμε να χρησιμοποιούμε batches με διαφορετικό μέγεθος κάθε φορά που τρέχει το πρόγραμμα), υλοποιούμε forward_pass και backprop όπου ενημερώνονται οι πίνακες των σφαλμάτων/παραγώγων.

Έπειτα σύμφωνα με τους τύπους ενημερώνουμε τα βάρη και τις πολώσεις βρίσκουμε το συνολικό σφάλμα της εποχής και το τυπώνουμε.

7) Αφού τερματίσει η εκπαίδευση ελέγχουμε την ικανότητα γενίκευσης του προγράμματος υλοποιώντας με τα πλέον ανανεωμένα βάρη την `forward_pass` ,για κάθε παράδειγμα του συνόλου ελέγχου, η οποία μας δίνει έναν πίνακα από τον οποίο λαμβάνουμε το στοιχείο με την μέγιστη τιμή όπου και αυτό το στοιχείο είναι και η κατηγορία στην οποία ταξινομείται. Συγκρίνουμε το αποτέλεσμα για κάθε παράδειγμα με την επιθυμητή έξοδο του και ελέγχοντας αν συμβαδίζουν υπολογίζουμε το ποσοστό σωστών αποφάσεων στο σύνολο ελέγχου το όποιο και τυπώνουμε.

3. Μελέτη του προβλήματος ταξινόμησης ΣΔΤ

Εξέταση και καταγραφή αποτελεσμάτων:
(μέσος όρος 10 εκτελέσεων για κάθε συνδιασμό)

Για διάφορους συνδυασμούς τιμών H1 H2 H3	PT2 B = 20 H1 = Tanhu H2 = Relu	PT3 B = 20 H1,2 = Tanhu H3 = relu
H1 = 8 H2 = 4 H3= 3 (for PT3 only)	% επιτυχίας 71.6	% επιτυχίας 68.7
H1 = 10 H2 = 5 H3= 5(for PT3 only)	% επιτυχίας 80.1	% επιτυχίας 76.3
H1 = 20 H2 = 10 H3= 10 (for PT3 only)	% επιτυχίας 94.6	% επιτυχίας 88.1
H1 =8 H2 =15 H3= 15(for PT3 only)	% επιτυχίας 93.6	% επιτυχίας 91.2

Για διαφορετικές συναρτήσεις ενεργοποίησης	PT2 B = 20 H1 = 20 H2 = 10	PT3 B = 20 H1 = 20 H2,3 = 10
1 Tanhu 2 Relu (PT2) 1,2 Tanhu 3 Relu (PT3)	% επιτυχίας 94.6	% επιτυχίας 88.1
1 ,2 Tanhu (PT2) 1,2,3 Tanhu (PT3)	% επιτυχίας 95.6	% επιτυχίας 94.6

Για διαφορετικού μεγέθους batches	PT2 H1,2 = Tanhu H1 = 20 H2 = 10	PT3 H1,2,3 = Tanhu H1 = 20 H2,3 = 10
B = N/20	% επιτυχίας 95.01	% επιτυχίας 93.9
B = N/200	% επιτυχίας 95.6	% επιτυχίας 94.6

Εκτύπωση Παραδειγμάτων: Ενδεικτικά screenshots

```
135. X1 = -0.133783,X2 = -0.082777 Cl = 2 (+) ||
136. X1 = 0.369292,X2 = 0.349040 Cl = 1 (+) ||
137. X1 = 0.759554,X2 = 0.087913 Cl = 2 (+) ||
138. X1 = 0.582753,X2 = 0.432015 Cl = 1 (+) ||
139. X1 = -0.069378,X2 = 0.517092 Cl = 3 (-) ||
140. X1 = -0.513456,X2 = 0.893366 Cl = 0 (+) ||
141. X1 = 0.042084,X2 = -0.120359 Cl = 3 (+) ||
142. X1 = 0.288540,X2 = -0.816554 Cl = 1 (+) ||
143. X1 = 0.097717,X2 = -0.648462 Cl = 1 (+) ||
144. X1 = -0.621332,X2 = -0.839920 Cl = 1 (+) ||
145. X1 = -0.627532,X2 = -0.336540 Cl = 0 (+) ||
146. X1 = 0.491537,X2 = -0.596994 Cl = 1 (+) ||
147. X1 = 0.474884,X2 = 0.219563 Cl = 1 (+) ||
148. X1 = 0.391705,X2 = -0.722797 Cl = 1 (+) ||
149. X1 = -0.458287,X2 = 0.987443 Cl = 3 (+) ||
150. X1 = -0.644901,X2 = 0.169150 Cl = 1 (+) ||
151. X1 = -0.703328,X2 = -0.209626 Cl = 0 (+) ||
152. X1 = -0.815114,X2 = -0.705545 Cl = 1 (+) ||
153. X1 = -0.977380,X2 = -0.652930 Cl = 2 (+) ||
154. X1 = 0.810641,X2 = -0.029832 Cl = 3 (+) ||
155. X1 = 0.364010,X2 = -0.273865 Cl = 0 (+) ||
156. X1 = -0.362324,X2 = -0.189505 Cl = 0 (+) ||
157. X1 = 0.839819,X2 = 0.567438 Cl = 0 (+) ||
158. X1 = -0.980946,X2 = -0.227337 Cl = 2 (+) ||
159. X1 = 0.554200,X2 = 0.008934 Cl = 3 (-) ||
160. X1 = -0.486510,X2 = 0.811960 Cl = 0 (+) ||
161. X1 = -0.756565,X2 = 0.932806 Cl = 3 (+) ||
162. X1 = 0.714814,X2 = 0.852731 Cl = 0 (+) ||
163. X1 = 0.212035,X2 = 0.834540 Cl = 0 (+) ||
164. X1 = -0.123259,X2 = -0.394970 Cl = 0 (+) ||
165. X1 = 0.200994,X2 = 0.447797 Cl = 1 (+) ||
166. X1 = 0.698997,X2 = 0.866213 Cl = 0 (+) ||
167. X1 = 0.112097,X2 = 0.622512 Cl = 0 (+) ||
168. X1 = -0.589627,X2 = 0.062495 Cl = 1 (+) ||
169. X1 = 0.899361,X2 = -0.913398 Cl = 3 (+) ||
170. X1 = -0.957941,X2 = 0.693000 Cl = 3 (+) ||
171. X1 = -0.892726,X2 = 0.744334 Cl = 3 (+) ||
```

```

1. X1 = -0.249514,X2 = -0.752951 Cl = 1 (+) ||
2. X1 = 0.084788,X2 = -0.493531 Cl = 1 (-) ||
3. X1 = 0.739681,X2 = -0.537924 Cl = 1 (+) ||
4. X1 = -0.832594,X2 = -0.546286 Cl = 1 (+) ||
5. X1 = 0.543177,X2 = -0.062875 Cl = 3 (-) ||
6. X1 = 0.352372,X2 = 0.613641 Cl = 0 (+) ||
7. X1 = -0.404982,X2 = -0.736937 Cl = 1 (+) ||
8. X1 = -0.720061,X2 = 0.192181 Cl = 1 (+) ||
9. X1 = -0.209749,X2 = -0.763063 Cl = 1 (+) ||
10. X1 = 0.981407,X2 = 0.572667 Cl = 2 (+) ||
11. X1 = 0.612269,X2 = -0.569440 Cl = 1 (+) ||
12. X1 = -0.794594,X2 = 0.265335 Cl = 1 (+) ||
13. X1 = 0.079415,X2 = 0.109041 Cl = 2 (+) ||
14. X1 = -0.511578,X2 = 0.536145 Cl = 0 (+) ||
15. X1 = 0.813382,X2 = -0.720562 Cl = 1 (+) ||
16. X1 = -0.590255,X2 = 0.503019 Cl = 1 (-) ||
17. X1 = -0.857874,X2 = 0.094543 Cl = 3 (+) ||
18. X1 = -0.522675,X2 = 0.560369 Cl = 0 (+) ||
19. X1 = 0.777800,X2 = 0.579807 Cl = 0 (+) ||
20. X1 = 0.245534,X2 = -0.910166 Cl = 3 (+) ||
21. X1 = -0.173351,X2 = -0.756736 Cl = 1 (+) ||
22. X1 = -0.741889,X2 = 0.601490 Cl = 0 (+) ||
23. X1 = 0.565963,X2 = 0.088715 Cl = 1 (+) ||
24. X1 = -0.234794,X2 = 0.956223 Cl = 3 (+) ||
25. X1 = 0.857814,X2 = 0.931948 Cl = 2 (+) ||
26. X1 = -0.717975,X2 = -0.515433 Cl = 1 (+) ||
27. X1 = -0.622126,X2 = 0.068170 Cl = 1 (-) ||
28. X1 = -0.940383,X2 = 0.192609 Cl = 3 (+) ||
29. X1 = -0.997543,X2 = 0.798664 Cl = 3 (+) ||
30. X1 = 0.242226,X2 = -0.294704 Cl = 0 (+) ||
31. X1 = -0.722206,X2 = 0.964407 Cl = 3 (+) ||
32. X1 = -0.975152,X2 = -0.049667 Cl = 2 (+) ||
33. X1 = 0.971396,X2 = -0.213455 Cl = 3 (+) ||
34. X1 = -0.473932,X2 = 0.185146 Cl = 1 (+) ||
35. X1 = 0.824130,X2 = 0.963717 Cl = 2 (+) ||

```

Από την μελέτη του προβλήματος για τις τιμές το PT3 φαίνεται να μην υπερτερεί του PT2, συγκεκριμένα παρατηρείται πως για λιγότερους νευρώνες στα επίπεδα το PT2 υπερτερεί σημαντικά ενώ όσο αυξάνονται οι νευρώνες το PT3 τείνει όλο και

περισσότερο στην γενικευτική ικανότητα του PT2. Αν χρειάζεται να το αιτιολογήσουμε θα λέγαμε πως το πρόβλημα είναι ότι οι διαδρομές εισόδων βαρών είναι περισσότερες έτσι κατάλήγει συνήθως το PT3 σε μεγαλύτερα τοπικά ελάχιστα κατά το gradient descent ανάλογα και με το μέγεθος του batch.

4. Αναφορές – Πηγές που χρησιμοποιήθηκαν κατά την εκπόνηση της εργασίας:

- Διαφάνειες στην ιστοσελίδα του μαθήματος (<https://www.cs.uoi.gr/~arly/courses/nn/slides>)
- Βοηθητικά search σε ιστοσελίδες για καλύτερη κατανόηση των δεδομένων της εργασίας, των ζητούμενων , την χρήση του mlr και σύνταξης ή βελτίωση του κώδικα και της γλώσσας (YouTube,GeeksForGeeks,StackOverflow)